

```
pip install datasets
```

```

Requirement already satisfied: datasets in /usr/local/lib/python3.11/dist-packages (2.14.4)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist-packages (from datasets) (2.0.2)
Requirement already satisfied: pyarrow>=8.0.0 in /usr/local/lib/python3.11/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.8,>=0.3.0 in /usr/local/lib/python3.11/dist-packages (from datasets) (0.3.7)
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.11/dist-packages (from datasets) (2.32.3)
Requirement already satisfied: tqdm>=4.62.1 in /usr/local/lib/python3.11/dist-packages (from datasets) (4.67.1)
Requirement already satisfied: xxhash in /usr/local/lib/python3.11/dist-packages (from datasets) (3.5.0)
Requirement already satisfied: multiprocessing in /usr/local/lib/python3.11/dist-packages (from datasets) (0.70.15)
Requirement already satisfied: fsspec>=2021.11.1 in /usr/local/lib/python3.11/dist-packages (from fsspec[http]>=2021.11.1->datasets) (2025.10.2)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.11/dist-packages (from datasets) (3.11.15)
Requirement already satisfied: huggingface-hub<1.0.0,>=0.14.0 in /usr/local/lib/python3.11/dist-packages (from datasets) (0.33.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from datasets) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-packages (from datasets) (6.0.2)
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (2.6.1)
Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (25.3.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (1.7.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (6.6.3)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (0.3.2)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (1.20.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from huggingface-hub<1.0.0,>=0.14.0->datasets) (3.16.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub<1.0.0,>=0.14.0->datasets) (4.12.2)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.2 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub<1.0.0,>=0.14.0->datasets) (1.2.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->datasets) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->datasets) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->datasets) (2.3.1)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->datasets) (2025.10.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas->datasets) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)

```

```
!pip install -U datasets transformers torchvision
```



Found existing installation: torchvision 0.21.0+cu124

Uninstalling torchvision-0.21.0+cu124:

Successfully uninstalled torchvision-0.21.0+cu124

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the s
gcsfs 2025.3.2 requires fsspec==2025.3.2, but you have fsspec 2025.3.0 which is incompatible.
torchaudio 2.6.0+cu124 requires torch==2.6.0, but you have torch 2.7.1 which is incompatible.
fastai 2.7.19 requires torch<2.7,>=1.10, but you have torch 2.7.1 which is incompatible.
Successfully installed datasets-4.0.0 fsspec-2025.3.0 nvidia-cublas-cu12-12.6.4.1 nvidia-cuda-cupti-cu12-12.6.80 nvidia-cuda-nvrtc

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import transforms
from datasets import load_dataset
from transformers import ViTForImageClassification, ViTFeatureExtractor
from torch.utils.data import DataLoader, Dataset
import torch.nn.functional as F
from torch.cuda.amp import GradScaler, autocast

from datasets import load_dataset
from transformers import ViTFeatureExtractor
from torchvision import transforms

# Load dataset without streaming
dataset = load_dataset("funkepal/medicinal_plant_images") # Removed streaming=True

# Load ViT feature extractor
feature_extractor = ViTFeatureExtractor.from_pretrained("google/vit-base-patch16-224")

# Define image transform
transform = transforms.Compose([
    transforms.Resize((224, 224)), # Resize to 224x224 for ViT
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.Lambda(lambda img: img.convert("RGB")), # Ensure RGB
    transforms.ToTensor(),
    transforms.Normalize(mean=feature_extractor.image_mean, std=feature_extractor.image_std)
])
```

🔄 /usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as :
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
README.md: 1.43k/? [00:00<00:00, 133kB/s]

train-00000-of-00001.parquet: 100% 250M/250M [00:03<00:00, 61.7MB/s]
test-00000-of-00001.parquet: 100% 60.3M/60.3M [00:00<00:00, 60.6MB/s]
Generating train split: 100% 4796/4796 [00:01<00:00, 5436.98 examples/s]
Generating test split: 100% 1199/1199 [00:00<00:00, 3651.41 examples/s]
preprocessor_config.json: 100% 160/160 [00:00<00:00, 2.62kB/s]
/usr/local/lib/python3.11/dist-packages/transformers/models/vit/feature_extraction_vit.py:30: FutureWarning: The class ViTFeatureExt
warnings.warn(

Custom dataset class to handle transformations
class PlantDataset(Dataset):
 def __init__(self, data, transform):
 self.data = data
 self.transform = transform

 def __len__(self):
 return len(self.data)

 def __getitem__(self, idx):
 item = self.data[idx]
 image = item['image']
 label = item['label']

 # Apply the transformation to the image
 image = self.transform(image)
 return image, torch.tensor(label, dtype=torch.long)

Apply the transformation using the custom dataset class

```
# Use the loaded dataset object (dataset_obj) instead of the string variable (dataset)
train_dataset = PlantDataset(dataset['train'], transform)
test_dataset = PlantDataset(dataset['test'], transform)

# DataLoader for batching and shuffling
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True) # Adjust batch size as needed
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```

```
# Define CNN for Feature Extraction
class CNNFeatureExtractor(nn.Module):
    def __init__(self):
        super(CNNFeatureExtractor, self).__init__()
        self.conv_layer = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )

    def forward(self, x):
        x = self.conv_layer(x)
        return x

# Instantiate CNN feature extractor
cnn_extractor = CNNFeatureExtractor()
```

```
# Vision Transformer for Classification
class HybridModel(nn.Module):
    def __init__(self):
        super(HybridModel, self).__init__()
        self.cnn = cnn_extractor
        self.vit = ViTForImageClassification.from_pretrained(
            "google/vit-base-patch16-224",
            num_labels=41, # Number of classes in your dataset
            ignore_mismatched_sizes=True # Add this line to ignore size mismatches
        )
        self.conv_proj = nn.Conv2d(128, 3, kernel_size=1) # Define the projection layer here

    def forward(self, x):
        # Extract features using CNN
        x = self.cnn(x)
        # Resize features to 224x224 for ViT using interpolation
        x = F.interpolate(x, size=(224, 224), mode='bilinear', align_corners=False)
        # Project features to 3 channels using a 1x1 convolution
        x = self.conv_proj(x) # Use the defined projection layer
        outputs = self.vit(pixel_values=x) # Use Vision Transformer for classification
        return outputs.logits
```

```
# Instantiate the model
model = HybridModel()

# Optimizer and Loss Function
optimizer = optim.Adam(model.parameters(), lr=1e-4)
criterion = nn.CrossEntropyLoss()

# Device configuration
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

```
# Mixed Precision Training
scaler = GradScaler() # Mixed precision scaler
```

```
↔ config.json: 69.7k/? [00:00<00:00, 771kB/s]

model.safetensors: 100% 346M/346M [00:09<00:00, 45.4MB/s]

Some weights of ViTForImageClassification were not initialized from the model checkpoint at google/vit-base-patch16-224 and are new!
- classifier.bias: found shape torch.Size([1000]) in the checkpoint and torch.Size([41]) in the model instantiated
- classifier.weight: found shape torch.Size([1000, 768]) in the checkpoint and torch.Size([41, 768]) in the model instantiated
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
/tmp/ipython-input-8-421820773.py:13: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler` instead.
  scaler = GradScaler() # Mixed precision scaler
```

```

# Training the Model
def train(model, train_loader, optimizer, criterion, device, scaler):
    model.train()
    total_loss = 0
    correct = 0

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad() # Clear gradients

        with autocast(): # Mixed precision
            outputs = model(images) # Forward pass
            loss = criterion(outputs, labels) # Calculate loss

        scaler.scale(loss).backward() # Backpropagation with scaled gradients
        scaler.step(optimizer) # Update weights
        scaler.update()

        total_loss += loss.item() # Accumulate loss
        correct += (outputs.argmax(1) == labels).sum().item() # Count correct predictions

    torch.cuda.empty_cache() # Free up GPU memory

    accuracy = correct / len(train_loader.dataset)
    return total_loss / len(train_loader), accuracy # Return average loss and accuracy

# Evaluation function
def evaluate(model, test_loader, criterion, device):
    model.eval()
    total_loss = 0
    correct = 0

    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images) # Forward pass
            loss = criterion(outputs, labels) # Calculate loss
            total_loss += loss.item()
            correct += (outputs.argmax(1) == labels).sum().item() # Calculate accuracy

    torch.cuda.empty_cache() # Free up GPU memory

    accuracy = correct / len(test_loader.dataset)
    return total_loss / len(test_loader), accuracy # Return average loss and accuracy

# Train and evaluate the model
num_epochs = 10
total_train_loss = 0
total_test_loss = 0

for epoch in range(num_epochs):
    print(f"Epoch {epoch + 1}/{num_epochs}")

    # Training phase
    train_loss, train_accuracy = train(model, train_loader, optimizer, criterion, device, scaler)
    print(f"Train Loss: {train_loss:.4f}, Train Accuracy: {train_accuracy:.4f}")

    # Testing phase
    test_loss, test_accuracy = evaluate(model, test_loader, criterion, device)
    print(f"Test Loss: {test_loss:.4f}, Test Accuracy: {test_accuracy:.4f}")

    # Accumulate losses (not accuracies) if you need them for reference
    total_train_loss += train_loss
    total_test_loss += test_loss

# Print final total loss and accuracy after all epochs
print(f"\nTotal Training Loss after {num_epochs} epochs: {total_train_loss:.4f}")
print(f"Total Test Loss after {num_epochs} epochs: {total_test_loss:.4f}")
print(f"Final Training Accuracy after {num_epochs} epochs: {train_accuracy:.4f}") # Final epoch accuracy
print(f"Final Test Accuracy after {num_epochs} epochs: {test_accuracy:.4f}") # Final epoch accuracy

```

↻ Epoch 1/10
 /tmp/ipython-input-9-3547069011.py:13: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast`
 with autocast(): # Mixed precision
 Train Loss: 2.2970, Train Accuracy: 0.4066
 Test Loss: 1.0880, Test Accuracy: 0.7148
 Epoch 2/10
 Train Loss: 0.6959, Train Accuracy: 0.8190

```

Test Loss: 0.4761, Test Accuracy: 0.8691
Epoch 3/10
Train Loss: 0.3591, Train Accuracy: 0.8999
Test Loss: 0.3925, Test Accuracy: 0.8966
Epoch 4/10
Train Loss: 0.2581, Train Accuracy: 0.9289
Test Loss: 0.3137, Test Accuracy: 0.9166
Epoch 5/10
Train Loss: 0.2203, Train Accuracy: 0.9435
Test Loss: 0.3188, Test Accuracy: 0.9199
Epoch 6/10
Train Loss: 0.1800, Train Accuracy: 0.9460
Test Loss: 0.2692, Test Accuracy: 0.9208
Epoch 7/10
Train Loss: 0.1746, Train Accuracy: 0.9516
Test Loss: 0.3270, Test Accuracy: 0.9083
Epoch 8/10
Train Loss: 0.1459, Train Accuracy: 0.9577
Test Loss: 0.2442, Test Accuracy: 0.9316
Epoch 9/10
Train Loss: 0.1447, Train Accuracy: 0.9595
Test Loss: 0.2273, Test Accuracy: 0.9349
Epoch 10/10
Train Loss: 0.1518, Train Accuracy: 0.9566
Test Loss: 0.2091, Test Accuracy: 0.9358

Total Training Loss after 10 epochs: 4.6274
Total Test Loss after 10 epochs: 3.8657
Final Training Accuracy after 10 epochs: 0.9566
Final Test Accuracy after 10 epochs: 0.9358

```

Start coding or [generate](#) with AI.

```

# Load the dataset from Hugging Face
from datasets import load_dataset
dataset = load_dataset("funkepal/medicinal_plant_images")

# Get the feature information (label mapping)
label_names = dataset['train'].features['label'].names

# Print the label names
print(f"\nThere are {len(label_names)} unique image class names:")
for idx, label in enumerate(label_names):
    print(f"Label {idx}: {label}")

# Optionally, inspect the structure of an individual sample (e.g., the first image and label)
first_sample = dataset['train'][0]
label_of_first_sample = label_names[first_sample['label']]

print(f"\nExample sample from dataset:")
print(f"Image: {first_sample['image']} \nLabel: {label_of_first_sample}")

# Count the occurrences of each class in the dataset using the label names
from collections import Counter

# Create a Counter for the labels (mapped to their names)
label_counts = Counter(label_names[item['label']] for item in dataset['train'])

print("\nLabel Distribution (Count of each class in the training set):")
for label, count in label_counts.items():
    print(f"{label}: {count} images")

```



```

Pomegranate: 114 images
Tulasi: 118 images
Curry_Leaf: 112 images
Wood_sorel: 112 images
Nooni: 120 images
Honge: 120 images
Jasmine: 151 images
Nagadali: 123 images
Bamboo: 108 images
Henna: 122 images
Gauva: 120 images
Pappaya: 113 images
Amruta_Balli: 116 images
Doddapatre: 114 images
Insulin: 114 images
Avacado: 124 images
Castor: 125 images
Rose: 137 images
Ashoka: 123 images
Raktachandini: 118 images
Lemon_grass: 120 images
Ekka: 114 images
Ashwagandha: 126 images
Betel: 118 images
Brahmi: 102 images
Basale: 118 images
Arali: 119 images
Mint: 117 images
Geranium: 117 images
Neem: 126 images
Nithyapushpa: 122 images
Aloevera: 126 images
Mango: 116 images
Betel_Nut: 120 images
Sapota: 115 images
Not_a_plant: 42 images

```

```

from PIL import Image
import torch
from torchvision import transforms
import matplotlib.pyplot as plt

# Preprocess function for the new image
def preprocess_image(image_path, feature_extractor):
    image = Image.open(image_path).convert('RGB') # Convert image to RGB

    # Define the transformations (resize, convert to tensor, normalize)
    transform = transforms.Compose([
        transforms.Resize((224, 224)), # Resize to the required input size for the model
        transforms.ToTensor(), # Convert the image to tensor
        transforms.Normalize(mean=feature_extractor.image_mean, std=feature_extractor.image_std)
    ])

    transformed_image = transform(image).unsqueeze(0) # Add batch dimension
    return transformed_image, image # Return both the tensor and original image for display

# ... (rest of your code)
# Function to classify an image using the model
def classify_image(model, image_tensor, device):
    model.eval() # Set the model to evaluation mode
    image_tensor = image_tensor.to(device) # Move the image tensor to the same device as the model

    with torch.no_grad():
        outputs = model(image_tensor) # Forward pass through the model
        # Directly use outputs without logits attribute
        predicted_class_idx = torch.argmax(outputs, dim=1).item() # Get the predicted class index

    return predicted_class_idx

# Load and preprocess the new image
image_path = '/content/aar2.jpg' # Specify the path to your new image
image_tensor, original_image = preprocess_image(image_path, feature_extractor)

# Load the model and move it to the same device (CPU or GPU)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# Classify the new image
predicted_class_idx = classify_image(model, image_tensor, device)

# Get the actual label name from the predicted class index
predicted_label = label_names[predicted_class_idx]

# Display the original image and the predicted class
plt.imshow(original_image)
plt.title(f"Predicted label: {predicted_label}")

```

```
plt.axis('off')
plt.show()

# Print the result
print(f"The model predicts the image is of: {predicted_label}")
```



Predicted Label: Aloevera



The model predicts the image is of: Aloevera

```
# Access the first item from the test dataset
test_image, true_label = test_dataset[20]

# Convert the tensor back to a PIL-compatible format for visualization
image_np = test_image.permute(1, 2, 0).numpy() # Convert CHW to HWC
image_np = (image_np * [0.229, 0.224, 0.225]) + [0.485, 0.456, 0.406] # Unnormalize
image_np = image_np.clip(0, 1) # Ensure pixel values are within [0, 1]

# Plot the image
import matplotlib.pyplot as plt
plt.imshow(image_np)
plt.axis("off")
plt.title(f"True Label: {true_label}")
plt.show()
```



True Label: 7



```
# Access the first item from the test dataset
test_image, true_label = test_dataset[20]

# Prepare the image for the model (add batch dimension)
test_image_batch = test_image.unsqueeze(0) # Shape: [1, C, H, W]

# Move the input to the same device as the model
test_image_batch = test_image_batch.to(device)

# Predict the label using the model
model.eval() # Set the model to evaluation mode
```

```
with torch.no_grad():
    output = model(test_image_batch)
    predicted_label = torch.argmax(output, dim=1).item() # Get the predicted class index
```

```
# Print the predicted label
print(f"Predicted Label: {predicted_label}")
print(f"True Label: {true_label}")
```

↗ Predicted Label: 7
True Label: 7

```
import matplotlib.pyplot as plt
from torchvision import transforms
```

```
# Access the first item from the test dataset
test_image, true_label = test_dataset[20]
```

```
# Prepare the image for the model (add batch dimension)
test_image_batch = test_image.unsqueeze(0) # Shape: [1, C, H, W]
```

```
# Move the input to the same device as the model
test_image_batch = test_image_batch.to(device)
```

```
# Predict the label using the model
model.eval() # Set the model to evaluation mode
with torch.no_grad():
    output = model(test_image_batch)
    predicted_label_index = torch.argmax(output, dim=1).item() # Get the predicted class index
```

```
# Get the predicted label name
predicted_label_name = label_names[predicted_label_index]
```

```
# Denormalize the image for visualization
# (Assuming you've defined 'mean' and 'std' for your normalization)
inv_normalize = transforms.Normalize(
    mean=[-m / s for m, s in zip(feature_extractor.image_mean, feature_extractor.image_std)],
    std=[1 / s for s in feature_extractor.image_std]
)
denormalized_image = inv_normalize(test_image)
```

```
# Convert the tensor back to a PIL-compatible format for visualization
image_np = denormalized_image.permute(1, 2, 0).cpu().numpy() # Convert CHW to HWC
image_np = image_np.clip(0, 1) # Ensure pixel values are within [0, 1]
```

```
# Plot the image
plt.imshow(image_np)
plt.axis("off")
plt.title(f"Predicted Label: {predicted_label_name}, True Label: {label_names[true_label.item()]}")
plt.show()
```

↗ Predicted Label: Bamboo, True Label: Bamboo



```
import matplotlib.pyplot as plt
from torchvision import transforms
```

```
# Step 5: Select a test image
test_image_info = test_dataset[20] # Use the first image in the test dataset
image = test_image_info[0] # Access the image using numerical index 0
label = test_image_info[1] # Access the label using numerical index 1
```

```
# The image is already a tensor, no need to apply transform again
```



```

input_image = image.unsqueeze(0) # Add batch dimension if needed

# Step 6: Predict the label
with torch.no_grad():
    output = model(input_image.to(device)) # Ensure input is on the correct device
    predicted_label = torch.argmax(output, dim=1).item()

# Step 7: Display the image and predicted label
# Denormalize the image before converting to PIL Image
mean = feature_extractor.image_mean
std = feature_extractor.image_std

# Create an inverse transform to reverse the normalization
inv_normalize = transforms.Normalize(
    mean=[-m / s for m, s in zip(mean, std)],
    std=[1 / s for s in std]
)

# Denormalize the image
denormalized_image = inv_normalize(image)

# Convert the denormalized image tensor to a PIL Image for display
image_to_display = transforms.ToPILImage()(denormalized_image)

plt.imshow(image_to_display)
plt.axis("off")
plt.title(f"Predicted Label: {predicted_label}, True Label: {label}")
plt.show()

```



Predicted Label: 7, True Label: 7



```

# Access the first item from the test dataset
test_image, _ = test_dataset[20]

# Convert the tensor back to a PIL-compatible format for visualization
image_np = test_image.permute(1, 2, 0).numpy() # Convert CHW to HWC
image_np = (image_np * [0.229, 0.224, 0.225]) + [0.485, 0.456, 0.406] # Unnormalize
image_np = image_np.clip(0, 1) # Ensure pixel values are within [0, 1]

# Plot the image
import matplotlib.pyplot as plt
plt.imshow(image_np)
plt.axis("off")
plt.show()

```



```
import matplotlib.pyplot as plt
from torchvision import transforms
from datasets import load_dataset # Import load_dataset
```

```
# Step 7: Test a single image from the test dataset
test_image, true_label = test_dataset[20] # Use the first image in the test dataset
test_image_batch = test_image.unsqueeze(0) # Add batch dimension
```

```
# Reload the dataset to get label names (or define them explicitly if known)
dataset = load_dataset("funkepal/medicinal_plant_images") # Reload dataset
label_names = dataset['train'].features['label'].names # Get label names
```

```
# Step 8: Predict the label
with torch.no_grad():
    # Move the input tensor to the same device as the model
    output = model(test_image_batch.to(device))
    predicted_label_index = torch.argmax(output, dim=1).item() # Get the predicted label index
    predicted_label_name = label_names[predicted_label_index] # Get the corresponding label name
```

```
# Step 9: Visualize the image and the predicted label
# Convert the tensor back to a PIL image for visualization
test_image_np = test_image.permute(1, 2, 0).numpy() # Convert CHW to HWC
test_image_np = (test_image_np * [0.229, 0.224, 0.225]) + [0.485, 0.456, 0.406] # Unnormalize
test_image_np = test_image_np.clip(0, 1)
```

```
plt.imshow(test_image_np)
plt.axis("off")
plt.title(f"Predicted Label: {predicted_label_name}, True Label: {label_names[true_label.item()]}") # Use label names for display
plt.show()
```



Predicted Label: Bamboo, True Label: Bamboo

